

电子科技大学
格拉斯哥学院
Glasgow College, UESTC

Team Design Project and Skills Individual Report

Project: Vortex Edge
Team: 71
Name: Chen YiChi
Student ID: [redacted]
Date: June 19, 2025

Autonomous UAV Navigation with Stereo-Inertial Fusion and Gradient-Based Local Planning

Yichi Chen, [redacted]

***Abstract*—This paper presents an end-to-end autonomy pipeline for small aerial vehicles integrating stereo–inertial state estimation, efficient local planning, and real-time flight control. By combining an Intel RealSense D430 depth camera with VINS-Fusion, EGO-Planner, and a PX4-based control interface, the proposed framework achieves precise localization, robust obstacle avoidance, and seamless trajectory execution in complex, GPS-denied environments. The system was implemented and tested in an indoor classroom environment, and results demonstrate reliable navigation, consistent target landing, and robust sensor integration. Detailed subsystem solutions, experimental validation, and analysis of system limitations and potential improvements are also provided.**

***Index Terms*—Visual–Inertial Odometry, Local Path Planning, PX4, RealSense D430, EGO-Planner, MAVROS, Autonomous UAV.**

I. Introduction

In recent years, Unmanned Aerial Vehicles (UAVs) have undertaken an increasingly significant role in multiple fields, including logistics transportation, infrastructure inspection, surveillance and reconnaissance, agricultural management, and emergency response. This trend largely owes to UAVs' high flexibility, operational efficiency, and adaptability. However, as mission complexity increases, the demands on autonomous navigation capabilities, precise positioning, robust obstacle avoidance, and reliable control systems also rise accordingly. These requirements underline the importance of integrated perception–planning–control frameworks that leverage visual–inertial sensors, local planners, and advanced flight-control platforms to achieve safe and efficient UAV operation.

Depth cameras are vital for UAV navigation, providing real-time, high-fidelity three-dimensional spatial data for environment mapping, obstacle detection, and avoidance. Among RGB-D sensors, the Intel RealSense D430 stands out due to its robust performance, compact form factor, cost-effectiveness, and extensive open-source SDK support. Recent comparative studies demonstrate its superior depth accuracy, spatial resolution, and measurement consistency, making it well suited for applications requiring precise close-range perception [1].

Accurate state estimation is critical for UAV stability and trajectory tracking, especially in dynamic and uncertain environments. Visual–Inertial Odometry (VIO) significantly improves navigation accuracy by fusing camera imagery with high-rate IMU measurements. VINS-Fusion employs a tightly coupled, keyframe-based optimization and loop-closure mechanism that integrates robust feature tracking and IMU pre-integration for online self-calibration of sensor extrinsics and biases. This approach maintains high-precision pose estimates under rapid maneuvers and visual degradations [2]. Furthermore, quantitative trajectory evaluation methods based on absolute and relative error metrics enable systematic benchmarking of VIO performance, providing objective guidance for system validation and optimization [3].

Local path planning is another core component, responsible for real-time obstacle avoidance and path optimization. Conventional planners often rely on full-field Euclidean Signed Distance Field (ESDF) computations, incurring significant memory and computational overhead. EGO-Planner introduces an ESDF-free gradient-based approach that locally computes obstacle gradients only along the candidate trajectory corridor, yielding collision-free polynomial paths with substantially reduced latency [4].

Ensuring real-time stability and precision requires a deterministic flight-control stack. PX4 is a node-based, multi-threaded open-source framework designed for deeply embedded autopilot applications. Its publish–subscribe middleware and POSIX-like API guarantee low-latency sensor-to-actuator loops, while its modular architecture accommodates integration with VIO and planning modules for complex autonomous missions [5].

Bridging high-level ROS-based development and low-level autopilot deployment, a Pixhawk–ROS co-development approach enables high-frequency control updates via MAVROS. Control algorithms can be designed and tested in ROS, then seamlessly communicate with PX4 for real-time flight execution, dramatically reducing integration complexity and risk [6].

In this work, I developed an integrated UAV autonomy pipeline that unified stereo–inertial state estimation, mapping, planning, and flight control. The main system modules were:

- **Stereo–Inertial State Estimation:** Accurate six-DoF localization was achieved using VINS-Fusion, which fused RealSense D430 stereo images and IMU data.
- **Environment Mapping:** Depth images were converted to a voxel grid map for local planning and obstacle avoidance.
- **Local Trajectory Planning:** Real-time, collision-free paths were generated with EGO-Planner.
- **Flight Control:** Planned waypoints were sent as PX4 setpoints via a custom `px4ctrl` ROS node.
- **Closed-Loop Execution:** MAVROS enabled real-time command execution and feedback for robust autonomous flight.

This architecture enabled reliable localization, efficient obstacle avoidance, and consistent mission execution in GPS-denied environments, as shown in Fig. 5. The following sections present implementation, testing, analysis, and future work.

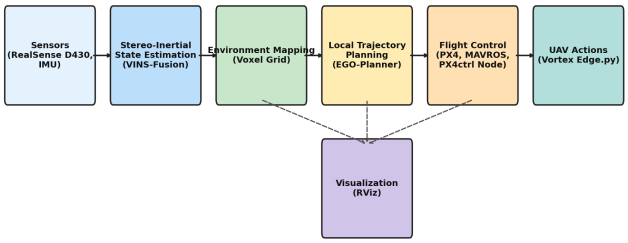


Fig. 1. System architecture of the integrated UAV autonomy pipeline developed in this work.

II. Experimental design-subsystem design and solutions

A. Establishing Data Pathways and Vision Nodes Between RealSense and UAVs

1) Environmental Preparation: On our Orange Pi running ROS, I installed the *librealsense2* SDK ,the *Vins-Fusion* and *realsense2_camera* ROS package, then plugged the RealSense D430 into a USB port.

2) One-Click Startup Script: To streamline the system initialization process, I developed a one-click startup script, `rspx4.sh`, which automatically launched all required nodes in the correct sequence. The script sequentially and automatically performed the following steps:

- **RealSense Driver Launch**: The script invoked `roslaunch realsense2_camera rs_camera.launch` to start the D430 camera node, which published color images, depth images, and point clouds.
- **Staggered Node Initialization**: Built-in `sleep` commands ensured that each node was given sufficient time to initialize and stabilize its topics before subsequent nodes were launched.
- **PX4 and MAVROS Startup**: The script started MAVROS and connected to the PX4 autopilot by running `roslaunch mavros px4.launch`, then sent initialization commands to set the appropriate flight mode and arm the vehicle.
- **VINS-Fusion Launch**: Finally, the script launched the VINS-Fusion pipeline via `roslaunch vins fast_drone_250.launch`, initializing the frontend, IMU pre-integration, and backend optimizer to publish odometry and point cloud data.

By running `sh rspx4.sh`, all camera drivers, MAVROS interfaces, and VINS-Fusion nodes were brought up in the proper order, enabling the complete vision-to-state estimation pipeline to be initialized with a single command. This automation significantly improved the efficiency, reproducibility, and reliability of the experimental setup.

B. Checking for VINS-Fusion Odometry Drift

After launching the vision nodes, I entered `rostopic echo /vins_fusion/imu_propagate` in the terminal. This command prints the IMU pre-integration outputs from VINS-Fusion in real time, including position, orientation, and covariance. At this stage, I gently moved the drone a short distance and then returned it to its original position, observing whether the position values in `/vins_fusion/imu_propagate` returned to their initial numbers.

- If the error when returning to the origin is within ± 0.5 m, it indicates that IMU pre-integration and pose fusion are functioning normally with no significant cumulative drift.
- If the drift exceeds 0.5 m, it means the state estimation has accumulated significant error. In this case, I stop the current script, restart it with `sh rspx4.sh`, and check the sensor calibration and node startup order.

C. Launching the Controller Node

After completing visual localization and state estimation, I proceeded to launch the autonomous flight controller node, which is responsible for the high-level motion control of the UAV. The process was as follows:

My teammate in charge of flight control found the open-source PX4ctrl code online and completed the necessary parameter modifications, such as PID gains, desired speeds, output limits, and motor control settings.

In this step, my main contribution was to modify the remapping rules in `run_ctrl.launch` so that the controller node received odometry input directly from the VINS-Fusion state estimation output, and published its flight control commands to the correct position command topic.

This ensured that the controller operated on the most accurate pose estimates and that all generated control signals were properly routed within the system.

After making these changes, I started the flight control node by running `roslaunch px4ctrl run_ctrl.launch` in the terminal. When the terminal began to display real-time status information such as UAV voltage, it indicated that the controller node was running successfully and had established communication with the flight controller and sensors, enabling autonomous flight capabilities. In our actual tests, this step went very smoothly, and we did not encounter any issues.

D. Launching the Ego-Planner Local Path Planner

After the flight controller node was successfully started, I proceeded to launch the local path planning module for autonomous obstacle avoidance and trajectory optimization, namely, the **Ego-Planner**.

My teammate in charge of the planning module sourced the open-source Ego-Planner code from GitHub—special thanks to the Fast Lab at Zhejiang University for their generous contribution of both the code and comprehensive documentation.

Within this open-source project, I mainly used the `single_run_in_exp.launch` file. This launch file is designed for experimental runs in real-world environments and is responsible for bringing up all the necessary ROS nodes and parameters for Ego-Planner, including map construction, sensor interface, depth data processing, and local trajectory optimization.

Moreover, during experiments, we could adjust key parameters in this launch file (such as map size, max velocity, max acceleration, etc.) to better adapt Ego-Planner to specific test environments and different UAV sensor configurations, thereby ensuring stable and safe algorithm performance.

My group member changed the parameters of the egoplanner (including `map_size`, `max_vel`, `max_acc`, and camera intrinsic parameters, etc.) in the `single_run_in_exp.launch` file according to the size of the test site, test safety, and accuracy.

After these preparations, I launched Ego-Planner by running:`roslaunch ego_planner single_run_in_exp.launch`

Once the terminal displays the log message "wait for goal or trigger", Ego-Planner is ready to perform real-time path planning and obstacle avoidance based on current perception and localization data.

E. Launching RViz Visualization to Monitor Flight and Grid Map

After all the core nodes on the UAV (perception, localization, control, and planning) had been successfully started, I opened a terminal and entered:

```
roslaunch ego_planner rviz.launch
```

At this point, the `rviz.launch` file would automatically start the RViz visualization tool and loaded the visualization configuration from Ego_planner.

RViz would display the grid map, real-time trajectory, obstacle point cloud, and flight pose data from the UAV, making it convenient to visually monitor the autonomous flight process and obstacle avoidance performance.

F. Developing and Running the Vortex Edge Autonomous Waypoint Program

After measuring the dimensions of the classroom, I planned a set of 3D waypoints based on the site layout and safety requirements, and saved them in the `point.txt` file. Subsequently, I developed the `VortexEdge.py` program, which enabled the UAV to autonomously complete the entire process of takeoff, waypoint navigation, and landing.

The overall workflow of the autonomous waypoint flight program is illustrated in Fig. 3 and can be summarized as follows:

- **Takeoff and Landing:** The system initially issued multiple takeoff commands to the PX4 controller, enabling the UAV to take off automatically. After the mission was completed, landing commands were sent to safely land the UAV.
- **Waypoint File Parsing:** All 3D waypoints (x, y, z, t) were loaded from a configuration file, allowing for flexible and repeatable mission planning.
- **Sequential Waypoint Execution:** The UAV visited each waypoint in sequence, sending the target position to the flight controller and pausing at each location for the preset duration.
- **Fully Autonomous Workflow:** Once started, the entire process—including takeoff, waypoint navigation, and landing—was executed automatically, without any further human intervention.

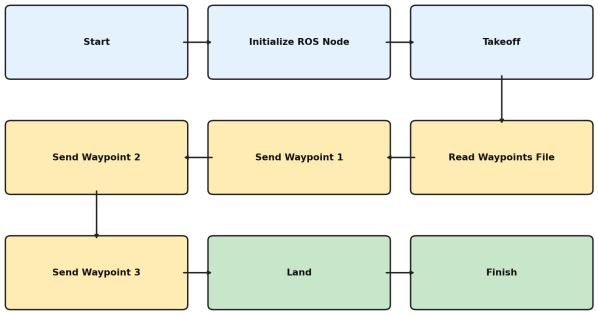


Fig. 2. Schematic flowchart illustrating the autonomous waypoint navigation process.

III. Result

A. USB Connection Stability and Camera Initialization(Related to Experimental Step A)

During early system deployment, I repeatedly encountered "control transfer returned error" messages when connecting the RealSense D430 camera to a USB 3.0 port on the Orange Pi. At first, I suspected the USB cable connection might be loose, so I reinforced both ends with glue to ensure a firm connection. However, this did not resolve the issue—the error persisted no matter how I adjusted the cable or port. After several unsuccessful attempts, I decided to try connecting the D430 to a USB 2.0 port instead. To my surprise, the camera worked flawlessly in this configuration, and all error messages disappeared. After researching relevant documentation, I believe the root cause is that USB 3.0's SuperSpeed link has much stricter requirements for cable quality, signal integrity, and driver support, which can easily cause control transfer timeouts. In contrast, the D430's data streams are more stable in the lower-speed mode of USB 2.0.

After running `sh rspx4.sh` and completing the initialization process, the terminal displayed a message such as "initialization finish." Subsequently, the system continuously printed lines like:

```
HP: requesting home position
```

This indicated that the onboard system was repeatedly requesting the home position from the flight controller, confirming that the camera node and core processes had been initialized and that communication with the PX4 autopilot was functioning properly.

B. State Estimation Accuracy: Visual Feature Dependence and Altitude Measurement(Related to Experimental Step B)

In practical tests, I found that when there were very few visual features in front of the RealSense D430 stereo camera—for example, when testing in an empty classroom—the position estimates from VINS-Fusion tended to drift much more significantly. Based on my observations, I analyzed that when there are few texture features and the visual constraints are

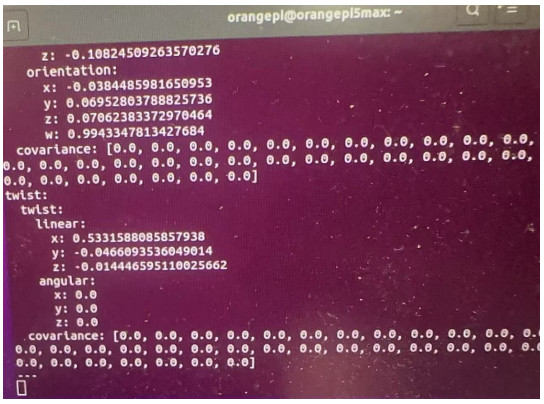
insufficient, VINS-Fusion is forced to rely more on IMU pre-integration, and the integration errors from the IMU accumulate rapidly, causing significant drift in localization.

In addition to this, I noticed another persistent issue: although the UAV's x and y positions remained stable, the z-axis coordinate was often nonzero, which puzzled me for quite some time. My initial hypothesis was that since our UAV only had a front-facing stereo camera, it could not obtain data for the area below the aircraft, which might lead to inaccurate altitude estimation. However, after further investigation, I discovered that the root cause was the flight controller's `EKF2_HGT_MODE` parameter being set to `barometer` by default, which meant that altitude (the z-axis) was being measured by the barometer rather than by the vision system. After I changed the `EKF2_HGT_MODE` parameter to `vision`, this issue was completely resolved, and the z-axis values became consistent with the UAV's actual altitude.

Upon successful launch of the VINS-Fusion node, the terminal output real-time stated estimation data, including fields such as:

- **Position** (e.g., `z: -0.000419...`)
- **Orientation** (quaternion values for `x`, `y`, `z`, `w`)
- **Covariance** (uncertainty values)
- **Twist** (linear and angular velocity)

These outputs demonstrated that VINS-Fusion was actively publishing state estimates, and that real-time IMU pre-integration and sensor fusion were working as expected.


orangepi@orangepi5max: ~
z: -0.10824509263570276
orientation:
x: -0.038448598165095 3
y: 0.06952803788825736
z: 0.07062383372970464
w: 0.9943347813427684
covariance: [0.0, 0.0, 0.0,

```
0.0    0.0    1.5    0.5
0.0    0.0    3.0    0.5
3.0    0.0    2.7    0.5
4.5    0.0    2.7    0.5
4.5   -5.5    2.7    0.5
1.8   -5.5    2.7    0.5
1.8   -5.5    0.5    0.5
```

That is, the UAV first took off and hovered at 1.5 meters, then ascended to 3.0 meters, descended to 2.7 meters, and then flew three straight segments and two turns at this height, finally landing next to the green area and black cross marked in the test site.

During the actual demonstration, I repeated the flight five times. Each time, the UAV was able to land within the green area and very close to the black cross. However, in two instances (including during the demo), when the UAV was set to fly at 2.7 meters, it actually flew about 20 centimeters lower than the preset altitude, with the actual height corresponding to the black line in the classroom.

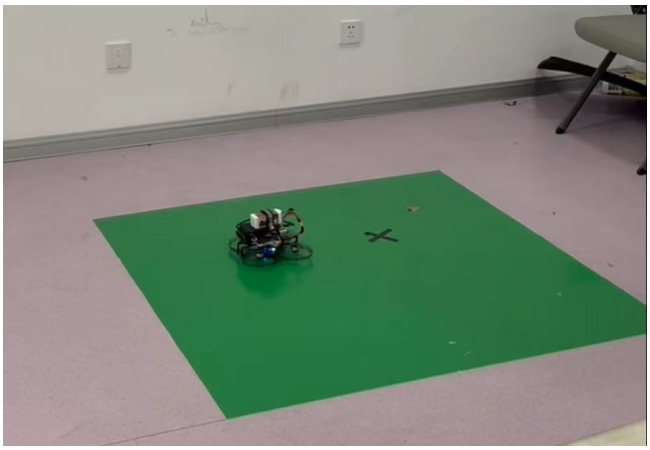


Fig. 5. Final UAV landing positions during repeated demonstration flights.

V. Analysis, Conclusions and Future Work

Analysis:

- **A. Camera Connection and Initialization:** The RealSense D430 camera exhibited frequent connection errors when using USB 3.0 ports, but operated reliably with USB 2.0. This suggests the camera's data transmission is sensitive to cable quality and port compatibility.
- **B. State Estimation and Sensor Fusion:** The accuracy of VINS-Fusion localization was strongly affected by the visual richness of the environment. In low-feature settings, the algorithm relied more heavily on IMU pre-integration, amplifying cumulative errors and leading to greater drift. Adjusting the flight controller's altitude estimation source from barometer to vision effectively resolved persistent z-axis drift and improved overall vertical accuracy.
- **C. Controller, Planning, and Visualization:** All modules—including the px4ctrl controller node, Ego-Planner, and RViz visualization—initialized and ran without technical issues. Each component provided the expected outputs and confirmed that the perception, planning, and control pipeline was functioning as intended.
- **D. Trajectory Following and Target Landing:** The UAV successfully completed the programmed waypoint sequence and repeatedly landed near the intended target, demonstrating robust performance in complex indoor environments. However, there remained minor altitude deviations (e.g., actual flight height was occasionally lower than intended), which may be attributed to sensor calibration, indoor air flow, or vision pipeline limitations.

Conclusions: The developed UAV system achieved reliable autonomous navigation, integrating perception, localization, planning, and control. The experimental results show strong robustness and repeatability, despite some small errors in altitude control and sensitivity to environmental features.

Future Work: To further improve z-axis accuracy, future work could involve adding a downward-facing camera or optical flow sensor for more robust altitude estimation. Additionally, upgrading to a more advanced depth camera, such as the Intel RealSense D435, would likely improve image quality and reduce susceptibility to environmental disturbances.

References

[1] M. Servi, A. Profili, R. Furferi, and Y. Volpe, "Comparative Evaluation of Intel RealSense D415, D435i, D455, and Microsoft Azure Kinect DK Sensors for 3D Vision Applications," *IEEE Access*, vol. 12, pp. 111311–111321, 2024, doi:10.1109/ACCESS.2024.3441238.

[2] T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," *IEEE Trans. Robotics*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018, doi:10.1109/TRO.2018.[redacted].

[3] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *Proc. 2018 IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Madrid, Spain, Oct. 2018, pp. 7244–7251, doi:10.1109/IROS.2018.8593941.

[4] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "EGO-Planner: An ESDF-Free Gradient-Based Local Planner for Quadrotors," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 478–485, Apr. 2021, doi:10.1109/LRA.2020.3047728.

[5] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in *Proc. 2015 IEEE Int. Conf. Robotics and Automation (ICRA)*, Seattle, WA, USA, May 2015, pp. 6235–6240, doi:10.1109/ICRA.2015.7140074.

[6] T. Chen, X. Zhuang, Z. Hou, and H. Chen, "A Pixhawk-ROS Based Development Solution for the Research of Autonomous Quadrotor Flight with a Rotor Failure," in *Proc. 2022 IEEE Int. Conf. Unmanned Systems (ICUS)*, Guangzhou, China, Apr. 2022, pp. 590–595, doi:10.1109/ICUS55513.2022.9986633.